

ARCHITECTURE GUIDE

Gym Platform Project Walkthrough

Complete project explanation for the Spring Boot backend in `D:\Codex-Gym`. This guide explains the purpose of each class, how the layers integrate, and how the important business flows move through the system.

Runtime Shape

Spring Boot API + PostgreSQL +
RabbitMQ + optional Email /
Telegram / WhatsApp channels.

Main Business Areas

Members, membership plans,
memberships, invoices,
payments, reminders, receipts,
and cards.

Design Style

Controller -> application service ->
repository / helper service ->
integration gateway.

Java 21 / 22 runtime compatible

Spring Boot 3.3

JPA + PostgreSQL

RabbitMQ notifications

Scheduler driven reminders

Telegram connect flow

1. Project In One Page

This project is a gym operations backend. It stores member and billing data in PostgreSQL, exposes REST APIs for frontends or operators, and sends notifications asynchronously through RabbitMQ. The code is intentionally layered so that HTTP handling, business rules, persistence, document generation, and external delivery are separate concerns.

Client / Frontend

- > REST Controllers
- > Application Services
 - > Repositories -> PostgreSQL
 - > Helper Services (mapping, code generation, document generation, message text generation)
 - > Application Events -> MemberCardAutomationListener
 - > NotificationPublisher -> RabbitMQ exchange -> queue -> NotificationConsumer
 - > EmailGateway / TelegramGateway / WhatsAppGateway

Scheduled jobs

- > FeeReminderScheduler
- > NotificationService
- > same RabbitMQ path as manual notifications

The notification system is asynchronous by design. The database decides what should be sent, `NotificationService` records a `PENDING` log, RabbitMQ transports the work, and `NotificationConsumer` performs the actual channel delivery.

2. Boot And Runtime Flow

Application start

- > `GymPlatformApplication`
 - > enables scheduling
 - > enables JPA auditing
- > `TimeConfiguration` creates `Clock` bean
- > `WebCorsConfig` loads CORS settings
- > `RabbitMqNotificationConfig` creates exchange / queue / DLQ / RabbitTemplate
- > Spring wires repositories, services, controllers, listeners, scheduler
- > `DevelopmentDataInitializer` seeds membership plans if database is empty
- > API becomes available on port 8080

Central runtime configuration comes from `src/main/resources/application.yml` and container overrides via environment variables.

3. End-To-End Business Flows

3.1 Member creation and automatic member card

```
POST /api/v1/members
-> MemberController.createMember
-> MemberService.createMember
    -> generate member code
    -> validate / normalize input
    -> decode profile image
    -> MemberRepository.save
    -> publish MemberCreatedEvent
-> transaction commits
-> MemberCardAutomationListener.onMemberCreated
-> NotificationService.queueMemberCard
    -> MemberCardService.prepareMemberCard
    -> MemberCardDocumentFactory.buildMemberCard
    -> create NotificationLog(PENDING)
    -> publish NotificationEventMessage to RabbitMQ
-> NotificationConsumer
    -> EMAIL or TELEGRAM channel gateway
    -> mark log SENT or FAILED
```

3.2 Membership creation and invoice generation

```
POST /api/v1/memberships
-> MembershipController.createMembership
-> MembershipService.createMembership
    -> load member
    -> load plan
    -> validate active member + active plan
    -> create Membership
    -> BillingService.createInvoice
        -> build membership signup invoice
    -> publish MembershipCardRefreshEvent
-> return MembershipEnrollmentResponse
-> after commit listener can queue updated membership card notification
```

3.3 Invoice payment and receipt notification

```
POST /api/v1/invoices/{invoiceId}/payments
-> InvoiceController.recordPayment
-> BillingService.recordPayment
    -> validate payment amount
    -> save Payment
    -> update Invoice amountPaid / balanceDue / status

Optional next step
POST /api/v1/invoices/{invoiceId}/receipt-notifications
-> NotificationService.queueInvoiceReceipt
    -> build receipt attachment with InvoiceDocumentFactory
    -> queue member and admin notifications
    -> publish through RabbitMQ

Alternative next step
POST /api/v1/invoices/{invoiceId}/receipt-notifications/pdf
-> upload an external PDF
-> NotificationService.queueInvoiceReceiptPdf
    -> send uploaded PDF instead of generated HTML receipt
```

3.4 Daily reminders

```
Scheduled cron (default 9:00 AM)
-> FeeReminderScheduler.pushScheduledReminders
-> MembershipService.refreshExpiredMemberships
```

```
-> BillingService.refreshOverdueInvoices
-> find due / overdue invoices
-> find active memberships ending soon
-> NotificationService.was...SentToday checks NotificationLogRepository
-> queueInvoiceReminder / queueRenewalReminder
-> publish to RabbitMQ
-> NotificationConsumer delivers channel messages
```

3.5 Telegram connect flow

```
POST /api/v1/members/{memberId}/telegram-connect
-> TelegramConnectionService.createConnectSession
  -> ensure Telegram is enabled and not dry-run
  -> create TelegramConnectionToken
  -> build deep link: https://t.me/{bot}?start=connect_{token}
```

```
Later sync step
POST /api/v1/members/telegram-connections/sync
-> TelegramConnectionService.syncConnections
  -> TelegramGateway.getUpdates
  -> parse /start connect payload
  -> find token
  -> ensure not expired / not consumed / no chat conflict
  -> save member.telegramChatId
  -> mark token consumed
  -> return sync report
```

4. Package Dependency Map

<code>com.codexgym.gym</code> GymPlatformApplication
<code>com.codexgym.gym.controller</code> Thin HTTP layer. Delegates to application services.
<code>com.codexgym.gym.application</code> Main business rules and orchestration. Coordinates repositories, helper services, events, and messaging.
<code>com.codexgym.gym.repository</code> Data access contracts backed by Spring Data JPA.
<code>com.codexgym.gym.entity</code> Persistent domain model for PostgreSQL.
<code>com.codexgym.gym.service</code> Stateless helpers for mapping, number generation, text generation, and document generation.
<code>com.codexgym.gym.messaging</code> RabbitMQ message models, publisher, and consumer.
<code>com.codexgym.gym.service.email / telegram / whatsapp</code> External delivery gateways.
<code>com.codexgym.gym.config</code> Beans, properties, scheduling support, RabbitMQ setup, CORS, seed data.
<code>com.codexgym.gym.scheduler</code> Time-driven business jobs.
<code>com.codexgym.gym.dto</code> Request and response models for the API.
<code>com.codexgym.gym.exception</code> Uniform API error handling.

5. Class Catalog: Boot, Config, And Runtime Files

Class / File	Why it exists	Main integrations
GymPlatformApplication	Application entry point. Enables scheduling and JPA auditing before Spring Boot starts the app.	Spring Boot, scheduler, auditing.
TimeConfiguration	Creates a shared <code>clock</code> bean so time-sensitive logic can be tested deterministically.	Used by billing, dashboard, membership, notifications, scheduler, Telegram link flow.
CorsProperties	Binds <code>gym.cors.allowed-origins</code> so CORS configuration is environment-driven.	<code>WebCorsConfig</code> , <code>application.yml</code> .
WebCorsConfig	Allows browser clients to call <code>/api/**</code> from configured origins.	Reads <code>CorsProperties</code> .
NotificationProperties	Central configuration model for scheduler windows, RabbitMQ names, brand name, and channel settings.	Used across messaging, scheduler, notifications, Telegram and WhatsApp delivery.
NotificationProperties. WhatsAppProperties	Stores WhatsApp enablement, dry-run flag, webhook URL, and bearer token.	<code>DelegatingWhatsAppGateway</code> , <code>WebhookWhatsAppGateway</code> .

<code>NotificationProperties</code> , <code>EmailProperties</code>	Stores email enablement, sender address, and admin recipient.	<code>SmtEmailGateway</code> , <code>NotificationService</code> .
<code>NotificationProperties</code> , <code>TelegramProperties</code>	Stores Telegram enablement, dry-run flag, bot token, bot username, admin chat, and connect-flow settings.	<code>TelegramBotGateway</code> , <code>TelegramConnectionService</code> .
<code>RabbitMqNotificationConfig</code>	Creates the RabbitMQ exchange, queue, dead-letter queue, bindings, JSON converter, and <code>RabbitTemplate</code> .	<code>RabbitTemplate</code> , <code>RabbitMQ</code> broker, <code>NotificationProperties</code> .
<code>DevelopmentDataInitializer</code>	Seeds default membership plans when the database is empty so local environments start with usable data.	Repositories, generators, <code>clock</code> .
<code>application.yml</code>	Main runtime configuration file. Declares datasource defaults, RabbitMQ connection settings, queue names, scheduler cron, and channel toggles.	Spring Boot configuration binding for the entire app.

6. Class Catalog: Domain Model

Class / Enum	Why it exists	Main integrations
<code>BaseEntity</code>	Gives every entity a UUID primary key plus created / updated timestamps through JPA auditing.	Inherited by all persistent entities.
<code>Member</code>	Represents a gym member and stores contact info, status, image, and Telegram identity.	Used by memberships, invoices, notifications, cards, Telegram link flow.
<code>MembershipPlan</code>	Stores reusable commercial plans such as duration, price, joining fee, and access level.	Selected during membership creation.
<code>Membership</code>	Represents a member's enrollment into a plan with start / end dates and active state.	Created and renewed by <code>MembershipService</code> ; linked to invoices.
<code>Invoice</code>	Stores billable financial documents for signup, renewal, or future add-ons.	Owned by <code>BillingService</code> ; linked to payments and reminders.
<code>Payment</code>	Represents money collected against an invoice.	Created by <code>BillingService</code> . <code>recordPayment</code> .
<code>NotificationLog</code>	Audit table for queued, sent, and failed notifications. Also used to suppress duplicate reminders on the same day.	<code>NotificationService</code> , scheduler, consumer.
<code>TelegramConnectionToken</code>	Temporary token used to safely link a Telegram private chat to a member.	<code>TelegramConnectionService</code> .
<code>MemberStatus</code>	Member lifecycle state.	Validated during membership creation and returned in APIs.
<code>MembershipStatus</code>	Membership lifecycle state.	Managed by <code>MembershipService</code> and dashboard metrics.
<code>InvoiceStatus</code>	Invoice lifecycle state such as pending, partially paid, paid, overdue, cancelled.	<code>BillingService</code> , dashboard, reminders.
<code>InvoiceType</code>	Explains what an invoice is for.	Used during signup and renewal invoice creation.
<code>PaymentMethod</code>	Captures how money was collected.	Used in payment requests and

		responses.
<code>NotificationChannel</code>	Declares the delivery medium: WhatsApp, Email, or Telegram.	Serialized inside queued notification messages.
<code>NotificationEventType</code>	Declares the business reason for a notification.	Stored in logs and transported through RabbitMQ.
<code>NotificationStatus</code>	Tracks whether a log is pending, sent, or failed.	Updated by <code>NotificationService.markDelivered</code> and <code>markFailed</code> .

7. Class Catalog: Repositories

Repository	Why it exists	Main custom queries
<code>MemberRepository</code>	Persists members and supports identity lookups.	Find by phone number, find by Telegram chat ID, count by status.
<code>MembershipPlanRepository</code>	Persists plan catalog.	Includes helper to list active plans by name.
<code>MembershipRepository</code>	Persists member enrollments.	Find active memberships by end date, latest membership for a member, count by status.
<code>InvoiceRepository</code>	Persists invoices and supports reminder selection.	Find open invoices by due date, count by status.
<code>PaymentRepository</code>	Persists payments and supports finance reporting.	List payments for invoice, sum collected revenue between dates.
<code>NotificationLogRepository</code>	Persists delivery audit records and prevents duplicate reminder sends.	Exists checks by invoice / membership and event type within a day window.
<code>TelegramConnectionTokenRepository</code>	Stores pending Telegram connection tokens.	Find by token, delete unconsumed tokens for a member.

8. Class Catalog: Application Services, Events, And Scheduler

Class	Why it exists	Main integrations
MemberService	Owns member creation, update, lookup, uniqueness handling, and profile-image decoding. Publishes the member-created event.	MemberRepository , MemberCodeGenerator , GymMapper , ApplicationEventPublisher .
MembershipPlanService	Owns plan creation, listing, and plan lookup.	MembershipPlanRepository , GymMapper .
MembershipService	Owns membership signup, renewal, expiry refresh, and membership counts. Also creates signup / renewal invoices.	MemberService , MembershipPlanService , MembershipRepository , BillingService , events.
BillingService	Owns invoice creation, payment recording, overdue refresh, invoice statistics, and receivable totals.	InvoiceRepository , PaymentRepository , InvoiceNumberGenerator , GymMapper .
DashboardService	Aggregates member, membership, invoice, and payment statistics into a single dashboard response.	MemberRepository , MembershipService , BillingService , PaymentRepository .
NotificationService	Core notification orchestrator. Builds business messages, creates notification logs, decides channels, and publishes queue messages.	Billing, membership, member, card service, repositories, document factories, message factory, publisher, config.
TelegramConnectionService	Implements the Telegram deep-link connect flow, manual linking, and update synchronization from the bot.	MemberService , MemberRepository , TelegramConnectionTokenRepository , TelegramGateway .
MemberCardService	Prepares member or membership card attachments by combining member data, latest membership data, and document generation.	MemberService , MembershipService , MemberCardDocumentFactory .
MemberCardService. MemberCardSnapshot	Small helper record that bundles member, membership, and generated attachment so callers get one object back.	Used by controllers and NotificationService .
DelegatingWhatsAppGateway	Chooses between real webhook delivery and simulated logging delivery based on config.	LoggingWhatsAppGateway , optional WebhookWhatsAppGateway , NotificationProperties .
MemberCardAutomationListener	Runs after successful transactions and automatically queues member card deliveries.	MemberCreatedEvent , MembershipCardRefreshEvent , NotificationService .
FeeReminderScheduler	Daily time-driven job that refreshes stale states and queues reminders for invoices and memberships.	BillingService , MembershipService , NotificationService , NotificationProperties .
MemberCreatedEvent	Signals that a member was successfully created.	Published by MemberService ; consumed by MemberCardAutomationListener .
MembershipCardRefreshEvent	Signals that a membership change should refresh the card.	Published by MembershipService ; consumed by MemberCardAutomationListener .

9. Class Catalog: Messaging Layer

Class	Why it exists	Main integrations
<code>NotificationPublisher</code>	Abstraction for putting notification work onto an async transport.	Implemented by <code>RabbitMqNotificationPublisher</code> .
<code>RabbitMqNotificationPublisher</code>	Sends serialized <code>NotificationEventMessage</code> objects to the configured exchange and routing key.	<code>RabbitTemplate</code> , <code>NotificationProperties</code> .
<code>NotificationEventMessage</code>	Queue payload carrying the notification log id, recipient, event type, channel, subject, body, and optional attachment.	Created by <code>NotificationService</code> ; consumed by <code>NotificationConsumer</code> .
<code>NotificationAttachment</code>	Serializable wrapper for file name, content type, and raw bytes.	Used for receipts and cards in email / Telegram delivery.
<code>NotificationConsumer</code>	RabbitMQ listener that receives queued messages, dispatches the correct channel gateway, and marks logs sent or failed.	<code>@RabbitListener</code> , email / Telegram / WhatsApp gateways, <code>NotificationService</code> .

10. Class Catalog: Helpers And External Integrations

Class	Why it exists	Main integrations
<code>GymMapper</code>	Converts JPA entities into API response DTOs and converts profile-image bytes into a data URL for clients.	Used by most services.
<code>MemberCodeGenerator</code>	Creates human-readable member codes with date prefix and random suffix.	<code>MemberService</code> .
<code>InvoiceNumberGenerator</code>	Creates human-readable invoice numbers with date prefix and random suffix.	<code>BillingService</code> .
<code>NotificationMessageFactory</code>	Centralizes the human-facing text for reminders, receipts, renewals, and cards.	<code>NotificationService</code> , <code>NotificationProperties</code> .
<code>MemberCardDocumentFactory</code>	Generates a styled HTML member / membership card attachment. It is an HTML attachment, not a PDF generator.	<code>MemberCardService</code> , <code>NotificationProperties</code> .
<code>InvoiceDocumentFactory</code>	Generates a styled HTML receipt attachment for paid invoices.	<code>NotificationService</code> , <code>NotificationProperties</code> .
<code>EmailGateway</code>	Abstraction for sending email with or without attachments.	Implemented by <code>SmtpEmailGateway</code> .
<code>SmtpEmailGateway</code>	Sends email through Spring Mail and SMTP. Supports attachment delivery.	<code>JavaMailSender</code> , <code>MailProperties</code> , <code>NotificationProperties</code> .
<code>TelegramGateway</code>	Abstraction for Telegram operations: send message, send document, read updates.	Implemented by <code>TelegramBotGateway</code> .
<code>TelegramGateway</code> . <code>TelegramUpdatesResponse</code>	Maps Telegram's <code>getUpdates</code> HTTP response.	Used in <code>TelegramBotGateway</code> .
<code>TelegramGateway</code> . <code>TelegramUpdate</code>	Maps one Telegram update payload.	Used by <code>TelegramConnectionService</code> .

TelegramGateway . TelegramMessage	Maps the message portion inside an update.	Used by TelegramConnectionService .
TelegramGateway . TelegramChat	Maps chat metadata inside a Telegram message.	Used by TelegramConnectionService .
TelegramBotGateway	Calls Telegram Bot HTTP endpoints to send messages, send documents, and fetch updates.	RestClient , NotificationProperties .
TelegramBotGateway . TelegramSendMessageRequest	Internal request record for Telegram's sendMessage API.	Used only inside TelegramBotGateway .
WhatsAppGateway	Abstraction for WhatsApp delivery.	Implemented by logging and webhook variants.
LoggingWhatsAppGateway	Dry-run fallback that logs messages instead of sending them to a real provider.	Used by DelegatingWhatsAppGateway .
WebhookWhatsAppGateway	Real WhatsApp delivery adapter that posts JSON to a configured webhook.	RestClient , NotificationProperties .
WebhookWhatsAppGateway . WhatsAppWebhookRequest	Internal request payload for the configured WhatsApp webhook.	Used only inside WebhookWhatsAppGateway .

11. Class Catalog: Controllers

Controller	Why it exists	Main endpoints
MemberController	HTTP layer for member CRUD-style actions, card download / send, and Telegram linking.	POST /members , PUT /members/{id} , GET /members , GET /members/by-phone , card endpoints, Telegram endpoints.
MembershipPlanController	HTTP layer for creating and listing plan catalog entries.	POST /membership-plans , GET /membership-plans .
MembershipController	HTTP layer for creating, listing, renewing memberships, and card operations.	POST /memberships , GET /memberships , POST /memberships/{id}/renew , card endpoints.
InvoiceController	HTTP layer for invoice listing, payment recording, payment history, and receipt-notification sending.	GET /invoices , POST /invoices/{id}/payments , GET /invoices/{id}/payments , receipt notification endpoints.
NotificationController	HTTP layer for manual member announcements and admin test messages.	POST /notifications/announcements , POST /notifications/admin-test .
DashboardController	Single endpoint for dashboard summary metrics.	GET /dashboard .

12. Class Catalog: DTOs

DTO	Why it exists	Used by
CreateMemberRequest	Validated input contract for member creation and update, including optional image and Telegram fields.	MemberController -> MemberService .
CreateMembershipPlanRequest	Validated input contract for plan creation.	MembershipPlanController .
CreateMembershipRequest	Validated input for creating a membership and its opening invoice.	MembershipController .
DashboardResponse	Compact response shape for dashboard metrics.	DashboardController .
ErrorResponse	Uniform API error response shape.	GlobalExceptionHandler .
InvoiceResponse	API shape for invoice data.	BillingService , GymMapper .
MemberResponse	API shape for member data.	MemberService , TelegramConnectionService .
MembershipEnrollmentResponse	Bundles membership and invoice when signup or renewal completes.	MembershipService .
MembershipPlanResponse	API shape for plan data.	MembershipPlanService .
MembershipResponse	API shape for membership data.	MembershipService .
PaymentResponse	API shape for payment records.	BillingService .
RecordPaymentRequest	Validated input for recording a payment.	InvoiceController .

<code>RenewMembershipRequest</code>	Validated input for membership renewal and renewal invoice values.	<code>MembershipController</code> .
<code>SendAdminTestNotificationRequest</code>	Validated input for a manual admin test message.	<code>NotificationController</code> .
<code>SendAnnouncementRequest</code>	Validated input for sending an announcement to one member.	<code>NotificationController</code> .
<code>TelegramConnectSessionResponse</code>	Returns the generated deep-link session details for Telegram linking.	<code>MemberController</code> -> <code>TelegramConnectionService</code> .
<code>TelegramConnectionSyncResponse</code>	Reports how many Telegram updates were scanned and which members linked successfully.	<code>TelegramConnectionService</code> .
<code>TelegramConnectionSyncResponse</code> . <code>TelegramConnectionResult</code>	One row in the sync report describing the outcome for a single connect attempt.	Nested inside the sync response.
<code>UpdateMemberTelegramChatRequest</code>	Validated input for manual Telegram chat-id linking.	<code>MemberController</code> .

13. Class Catalog: Exception Handling And Tests

Class	Why it exists	Main coverage / behavior
<code>ApiException</code>	Business-friendly exception carrying an HTTP status plus message.	Thrown by services whenever domain rules are violated.
<code>GlobalExceptionHandler</code>	Maps business, validation, and unexpected exceptions into consistent JSON error responses.	Handles <code>ApiException</code> , bean validation failures, generic errors.
<code>BillingServiceTest</code>	Verifies payment recording updates invoice balances and rejects overpayment.	Focuses on billing rule correctness.
<code>MembershipServiceTest</code>	Verifies membership creation also creates the correct invoice and publishes a card-refresh event.	Focuses on signup orchestration.
<code>NotificationServiceTest</code>	Verifies receipt queueing behavior, receipt attachment generation, and admin-copy suppression when chats match.	Focuses on notification orchestration.
<code>NotificationConsumerTest</code>	Verifies Telegram document dispatch when an attachment is present and that logs are marked delivered.	Focuses on channel dispatch behavior.

14. Integration Notes And Key Design Decisions

- **Why RabbitMQ exists:** the app does not send external messages directly inside core request paths. It queues work so APIs stay faster and external outages are isolated.
- **Why notification logs exist:** they provide an audit trail and allow "was this reminder already sent today?" checks.
- **Why a shared Clock bean exists:** business logic and tests can agree on time without calling `LocalDate.now()` everywhere.
- **Why document factories return HTML attachments:** the system currently generates styled HTML files for receipts and cards. A separate uploaded-PDF flow exists for invoice receipts when a real PDF file is available.
- **Why application events are used:** member-card delivery is triggered after a successful transaction commit, which avoids sending cards for rolled-back writes.
- **Why WhatsApp uses delegation:** environments without a real webhook still work through a dry-run logging adapter.
- **Why Telegram link tokens exist:** they prevent arbitrary chat-id assignment and create a safer "deep-link then confirm" integration flow.

15. Suggested Reading Order For A New Developer

1. GymPlatformApplication
2. application.yml
3. Domain entities: Member, MembershipPlan, Membership, Invoice, Payment, NotificationLog
4. Repositories
5. Application services:
 - MemberService
 - MembershipPlanService
 - MembershipService
 - BillingService
 - NotificationService
6. Messaging:
 - RabbitMqNotificationConfig
 - RabbitMqNotificationPublisher
 - NotificationConsumer
7. External gateways:
 - SmtpEmailGateway
 - TelegramBotGateway
 - DelegatingWhatsAppGateway
8. Controllers
9. Scheduler and TelegramConnectionService
10. Tests for confidence in the main orchestration paths

16. Final Mental Model

If you remember only one mental picture, make it this: the project is a CRUD-and-workflow backend where **memberships create invoices, payments update invoices, notifications are queued asynchronously**, and **cards / receipts are generated as attachments**. Controllers stay thin, application services own rules, repositories own persistence, and RabbitMQ separates business decisions from external message delivery.